King And Peasants

Ismael Carrasco Mkhazni

Trabajo fin de Grado

Supervisado por Dr. Pablo Trinidad Martín-Arroyo

Universidad de Sevilla

abril 2026

Publicado en abril 2026 por
Ismael Carrasco Mkhazni
Copyright © MMXXVI
http://www.lsi.us.es/~trinidad
ismcarmkh@alum.us.es

Pon aquí cuestiones acerca del copyright

Yo, D. Ismael Carrasco Mkhazni con NIF número 15456902M,

DECLARO

mi autoría del trabajo que se presenta en la memoria de este trabajo fin de grado que tiene por título:

King And Peasants

Lo cual firmo,

Fdo. D. Ismael Carrasco Mkhazni
en la Universidad de Sevilla
30/04/2026

Tu dedicatoria aquí

Agradecimientos

No olvides añadir una nota de agradecimiento a quienes hayan contribuido emocionalmente al proyecto fin de Grado.

Resumen

Durante la primera década del siglo XXI, los videojuegos de navegador representaron la principal puerta de entrada al ocio digital para una generación de jóvenes, democratizando el entretenimiento gracias a su inmediatez y accesibilidad. Plataformas impulsadas por tecnologías como *Adobe Flash* permitieron que millones de usuarios disfrutaran de experiencias multijugador sencillas y directas, sin barreras de entrada ni hardware costoso. Sin embargo, los habitos de consumo fueron cambiando progresivamente debido al desplazamiento de la industria hacia grandes producciones y ecosistemas móviles cerrados, culminando con el cese definitivo del soporte de Flash Player en 2020 [1]. El presente Trabajo Fin de Grado, *King and Peasant*, nace de la motivación personal de recuperar esa esencia nostálgica, reconstruyendo la experiencia del juego social de navegador bajo los estándares de la ingeniería de software moderna. El objetivo no es competir con la industria comercial, sino crear un tributo tecnológico que devuelva al usuario la posibilidad de .entrar y jugarçon amigos de forma instantánea.

Para conseguir este objetivo, se ha adaptado al entorno web el juego de cartas estratégico asimétrico *King and Peasant* mediante un ciclo de desarrollo ágil basado en Scrum. Dicha implementación se ha llevado a cabo empleando tecnologías modernas como React, Express, Node.js, MariaDB y WebSockets, bajo el patrón arquitectónico Modelo-Vista-Controlador (MVC). Como resultado, se ha desplegado una aplicación web funcional y dockerizada que integra un sistema de gestión de usuarios, salas de espera multijugador (lobbies) y un motor de reglas en tiempo real, demostrando que es posible utilizar las tecnologías web actuales para ofrecer una experiencia interactiva y fluida, apoyada en una base técnica sólida y escalable que cumple con los estándares de la ingeniería del software profesional.

Abstract

During the first decade of the 21st century, browser games represented the main gateway to digital leisure for a generation of young people, democratizing entertainment thanks to their immediacy and accessibility. Platforms driven by technologies such as *Adobe Flash* allowed millions of users to enjoy simple and direct multiplayer experiences, without entry barriers or expensive hardware. However, consumption habits progressively changed due to the industry's shift towards large-scale productions and closed mobile ecosystems, culminating in the definitive end of support for Flash Player in 2020 [1]. The present Bachelor's Thesis, *King and Peasant*, arises from the personal motivation to recover that nostalgic essence, reconstructing the social browser game experience under the standards of modern software engineering. The objective is not to compete with the commercial industry, but to create a technological tribute that gives back to the user the ability to "jump in and play"with friends instantly.

To achieve this goal, the asymmetric strategic card game *King and Peasant* has been adapted to the web environment through an agile development cycle based on Scrum. This implementation has been carried out using modern technologies such as React, Express, Node.js, MariaDB, and WebSockets, under the Model-View-Controller (MVC) architectural pattern. As a result, a functional and dockerized web application has been deployed, integrating a user management system, multiplayer waiting rooms (lobbies), and a real-time rules engine, demonstrating that it is possible to use current web technologies to offer an interactive and fluid experience, supported by a solid and scalable technical foundation that meets the standards of professional software engineering.

Índice general

I Introducción **1**

1. Contexto **3**

1.1. Motivación del Proyecto 4

1.2. Análisis de viabilidad y acotación del alcance 4

1.3. Desafíos de ingeniería y retos del desarrollo 5

1.4. Estructura de la Memoria 5

2. Objetivos **7**

2.1. Definición y delimitación del alcance 8

2.1.1. Dentro del alcance 8

2.1.2. Fuera del alcance 8

2.2. Objetivo General 9

2.3. Desglose de objetivos técnicos y metodológicos 9

II Organización del proyecto **11**

3. Estado del Arte y Marco Tecnológico **13**

3.1. Análisis de soluciones existentes y estado del arte 14

3.2. Arquitectura de comunicación y sincronización 15

3.3. Tecnologías de *Backend* y Ecosistema de Desarrollo 16

3.4. Tecnologías de Frontend Reactividad de Interfaz 16
3.5. Gestión de la persistencia de datos 17
3.6. Contenedorización y automatización de pruebas 18

4. Metodología **19**
4.1. Estructura organizacional del proyecto 20
4.2. Metodología de desarrollo 20
4.2.1. Análisis de alternativas tradicionales 21
4.2.2. Elección: Metodología Ágil (*Scrum*) 21
4.3. Adaptación al proyecto: El ciclo de vida ágil 22
4.3.1. Sprints Semanales 22
4.3.2. Coordinación y herramientas de gestión 23

5. Planificación **25**
5.1. Resumen temporal del proyecto 26
5.2. Planificación inicial 26
5.2.1. Cronograma de iteraciones (Nivel Micro) 27
5.3. Gestión de desviaciones y medidas correctivas 29
5.3.1. Incidencia técnica: Fallo en el hardware de desarrollo 29
5.3.2. Conciliación académica y personal 29

6. Costes **31**
6.1. Resumen de costes del proyecto 32
6.2. Costes de personal 32
6.3. Costes materiales 32
6.4. Costes indirectos 32

III Desarrollo del proyecto 33

7. Análisis y Diseño del Sistema 35

7.1. Análisis de Requisitos 36

7.1.1. Requisitos Funcionales y Actores 36

7.1.2. Requisitos No Funcionales (RNF) 36

7.2. Modelado del Sistema 37

7.2.1. Diagrama de Casos de Uso 37

7.2.2. Comportamiento Dinámico: Flujo de Ejecución 37

7.3. Lista de características 38

7.4. Diseño arquitectónico y Organización 39

7.4.1. Arquitectura del Backend (Servidor) 39

7.4.2. Arquitectura del Frontend (Cliente) 40

7.4.3. Infraestructura y Despliegue 41

8. Costes 43

8.1. Características a desarrollar 44

8.2. Diseño 44

8.3. Implementación 44

8.4. Pruebas 46

8.5. Despliegue 46

IV Cierre del proyecto 47

9. Manual de usuario 49

9.1. Sección libre 50

10. Conclusiones **51**

10.1. Informe post-mortem 52

10.1.1. Lo que ha ido bien 52

10.1.2. Lo que ha ido mal 52

10.1.3. Discusión 52

10.2. Trabajos futuros 52

V Appendices **53**

A. Software Product Lines **55**

A.1. Software Product Lines 56

A.2. Feature Models 56

A.3. Automated Analysis of Feature Models 58

A.3.1. Scope 58

A.4. Dynamic Software Product Lines (DSPL) 61

A.5. Hypothesis and Objectives 61

Referencias bibliográficas **64**

Índice de figuras

3.1. Interfaz de *Tabletop Simulator*, basada en la simulación física tridimensional sin validación de reglas. 14

3.2. Interfaz de *Board Game Arena*, basada en la automatización del estado de la partida en 2D. 15

7.1. Diagrama General de Casos de Uso del sistema. 37

7.2. Diagrama de Secuencia: Registro, creación de sala y flujo de partida. . . 38

8.1. Diagrama UML de diseño para la iteración 1 45

A.1. An example of a Home Integration System 57

A.2. A different view on AAFM distinguishing between information extraction and explanatory operations 59

Índice de cuadros

5.1. Tabla resumen de tiempos y planificación 26

5.2. Cronograma inicial de tareas distribuidas por Sprints semanales. 28

6.1. Tabla resumen de costes 32

8.1. Análisis de valor aportado 0001 44

8.2. Memorando técnico 0001 45

A.1. Most frequently used explanatory operations and their corresponding information extraction operations 63

Figura: Aquí el modelo de diseño en formato vectorial preferentemente (pdf) . . 45

To Abductive Section in 2.1 56

Figura: A feature model example 57

To Abductive Intro 58

1

Contexto

...

Ernest Hemingway (1899–1961),
Novelist

El siguiente capítulo aborda la concepción y el marco operativo del proyecto King and Peasant. Se expone la motivación inicial centrada en el desarrollo colaborativo y el interés por la arquitectura de videojuegos. Asimismo, se justifica la acotación del alcance del trabajo basada en un análisis de viabilidad técnica y temporal, fundamentando la decisión de adaptar un juego de mesa preexistente mediante tecnologías web modernas. Por último, se detallan los principales desafíos de ingeniería identificados, estableciendo la base para los objetivos y el desarrollo técnico que se describen en los capítulos posteriores.

El desarrollo de sistemas interactivos multijugador representa uno de los retos más dinámicos y exigentes dentro de la ingeniería de software contemporánea. Este proyecto, centrado en la adaptación digital del juego de mesa *King and Peasant*, surge como una respuesta técnica a la voluntad de crear experiencias de ocio accesibles y sociales bajo estándares profesionales. Lejos de ser un mero ejercicio de programación, el trabajo se concibe como un proyecto integral de ingeniería donde la gestión de recursos, la toma de decisiones estratégicas y la coordinación de un equipo de desarrollo son tan críticas como la implementación del código.

A continuación, se detalla el contexto que da origen a este Trabajo Fin de Grado (en adelante, TFG). Se comienza explorando las motivaciones que impulsan el desarrollo colaborativo, para posteriormente analizar la viabilidad del proyecto y los criterios técnicos que llevaron a definir su alcance actual. Finalmente, se presentan los desafíos de ingeniería que componen la solución propuesta, centrados en la transición del entorno físico al digital y la gestión de la concurrencia en tiempo real.

1.1 Motivación del Proyecto

La motivación principal detrás de la elección de este TFG es el interés por la arquitectura de videojuegos y el deseo de invertir el esfuerzo académico en un producto creativo y estimulante, resultando así intrínsecamente gratificante durante su ciclo de construcción. A su vez, dado el carácter aislado que suelen tener los proyectos académicos, se tomó la decisión de realizar el desarrollo como un esfuerzo mediante un equipo compuesto por dos personas. Al plantearse como un trabajo colaborativo, el proyecto se convierte en un ejercicio de coordinación técnica y trabajo en equipo, simulando las dinámicas y exigencias de un entorno profesional real en la industria del software.

1.2 Análisis de viabilidad y acotación del alcance

El planteamiendto inicial de diseñar y desarrollar íntegramente un videojuego original fue descartado en las fases preliminares. La inversión en horas del diseño desde cero de mecánicas, el balanceo de reglas y la curva de aprendizaje presente en los motores gráficos actuales excedía el alcance de un TFG. Dado que la viabilidad de un proyecto de ingeniería de software depende de la correcta alineación entre los objetivos y los recursos disponibles, se tomó la decisión de acotar el alcance funcional adoptando el motor de reglas de un juego de mesa físico ya existente, además de hacer uso de los

conocimientos adquiridos a lo largo de la carrera, en tecnologías de desarrollo web.

En consecuencia, se optó por la adaptación digital de *King and Peasant*, un juego de mesa de tipo *Print & Play* con reglas ya consolidadas creado por la empresa nPipsGames [12]. Esta decisión permitió desplazar el foco hacia la resolución de los retos tecnológicos de la implementación: la construcción de una arquitectura robusta, la gestión de la concurrencia en tiempo real y la coordinación de un equipo de desarrollo colaborativo utilizando el stack tecnológico ya dominado por los autores (React y Node.js)

1.3 Desafíos de ingeniería y retos del desarrollo

El desarrollo de este proyecto presenta tres desafíos técnicos y organizativos que justifican su complejidad. En primer lugar, la adaptación de un juego de mesa como *King and Peasant* conlleva una labor de abstracción de reglas. No solo se trata de digitalizar una interfaz, sino de transformar las mecánicas pensadas para la interpretación humana y física para que se integren en un sistema de estados robusto con la capacidad de validar acciones y prevenir comportamientos no permitidos de forma automatizada.

En segundo lugar, el componente multijugador añade el reto de la concurrencia y la sincronización de estados. Es necesario asegurar que todos los jugadores dispongan de la misma información de manera instantánea, al mismo tiempo que se garantiza el correcto manejo de múltiples eventos simultáneos, manteniendo la integridad del estado global de la partida.

Finalmente, la coordinación de un equipo de desarrollo supone un desafío organizativo adicional de colaboración. La gestión de un repositorio compartido, el uso de ramas de trabajo y la integración de módulos desarrollados en paralelo exigen un uso constante de metodologías de trabajo ágiles y herramientas de control de versiones para asegurar la coherencia del producto final.

1.4 Estructura de la Memoria

Por rellenar.

2 Objetivos

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Este capítulo establece el marco operativo y las metas fundamentales del TFG. En primer lugar, se delimita el alcance del proyecto, definiendo las responsabilidades técnicas asumidas y las restricciones funcionales del sistema. A continuación, se formula el objetivo general centrado en la adaptación digital de King and Peasant, para finalmente desglosarlo en un conjunto de objetivos específicos técnicos, de diseño y metodológicos que servirán como hitos medibles a lo largo del ciclo de desarrollo.

Tras establecer el contexto histórico y la motivación del proyecto en el primer capítulo, resulta imprescindible trazar una hoja de ruta técnica clara. Este segundo capítulo tiene como propósito fundamental acotar la magnitud del desarrollo y definir con precisión las metas del trabajo. Para ello, se comenzará delimitando el alcance funcional del sistema, diferenciando las competencias estrictamente ligadas a la ingeniería de software de aquellos elementos descartados por viabilidad temporal. Posteriormente, se formulará el objetivo general que compone la digitalización de *King and Peasant*, desglosándolo finalmente en una serie de hitos técnicos y metodológicos que guiarán las futuras fases de implementación y despliegue.

2.1 Definición y delimitación del alcance

La correcta delimitación del alcance es un factor crítico para asegurar la viabilidad del TFG. Dadas las limitaciones expuestas en el capítulo anterior, es necesario establecer ciertas fronteras en el desarrollo del proyecto. A continuación, se detallan los elementos que componen el alcance y aquellos que han sido explícitamente excluidos.

2.1.1 Dentro del alcance

El desarrollo abarca el ciclo de vida de la aplicación web, incluyendo:

- **Arquitectura de comunicación:** Implementación de un servidor capaz de gestionar sesiones concurrentes y comunicación en tiempo real mediante *WebSockets*.
- **Motor de reglas digital:** Traducción del reglamento de *King and Peasant* a una lógica de negocio que valide automáticamente los estados de la partida.
- **Gestión de usuarios:** Diseño de persistencia de datos para perfiles, amistades e histórico de partidas.
- **Despliegue:** Contenedorización del entorno de producción mediante *Docker* para garantizar su accesibilidad.

2.1.2 Fuera del alcance

Para centrar los esfuerzos en la calidad técnica del software, quedan excluidos:

- **Diseño original de mecánicas:** Se delega el balanceo y autoría del juego en el reglamento físico creado por nPipsGames.

- **Desarrollo móvil nativo:** El sistema se limita al entorno web, sin contemplar aplicaciones específicas para iOS o Android.
- **Monetización:** El producto se concibe como una herramienta de ocio gratuita, excluyendo pasarelas de pago o sistemas publicitarios.
- **Sistema competitivo (*Ranked*):** No se implementarán sistemas de emparejamiento basados en habilidad (*matchmaking*) ni puntuaciones de clasificación global (tipo Elo).
- **Economía virtual:** Queda excluida la creación de divisas internas (*in-game currency*) y sistemas de progresión basados en la adquisición de bienes virtuales.
- **Inteligencia Artificial (PvE):** El proyecto se centra estrictamente en la interacción jugador contra jugador (PvP), por lo que se descarta el desarrollo de agentes autónomos o *bots* contra los que jugar en solitario.

2.2 Objetivo General

El objetivo central de este TFG es el diseño, implementación y despliegue de la versión digital del juego asimétrico *King and Peasant* en una plataforma web multijugador. La meta final es ofrecer un ecosistema persistente que garantice la inmediatez de los juegos de navegador clásicos mediante el uso de tecnologías web modernas, una arquitectura cliente-servidor robusta y metodologías profesionales de desarrollo, asegurando una experiencia de usuario fluida, sincronizada y de alta disponibilidad.

2.3 Desglose de objetivos técnicos y metodológicos

Para alcanzar la meta global descrita anteriormente, el proyecto se desglosa en los siguientes objetivos específicos, los cuales actúan como hitos medibles del desarrollo:

- **Objetivo 1. Desarrollo de la lógica de juego y sincronización en tiempo real.** Implementar el motor de reglas del juego de mesa asegurando la integridad de la partida. Esto implica el uso de *WebSockets* para garantizar una comunicación bidireccional instantánea entre el servidor y los clientes, de modo que todos los jugadores perciban el mismo estado del tablero simultáneamente y sin latencia perceptible.

- **Objetivo 2. Construcción de un ecosistema social persistente.**
  Desarrollar un sistema de perfiles de usuario robusto que vaya más allá de la partida. Esto incluye el registro y autenticación segura de usuarios, gestión de listas de amigos, historial de partidas y estadísticas de rendimiento, fomentando la retención y la comunidad dentro de la plataforma.

- **Objetivo 3. Diseño de una Experiencia de Usuario (UX/UI) integral e intuitiva.**
  Implementar una interfaz gráfica limpia y ergonómica que traslade fielmente la fluidez del juego de mesa original al entorno digital. Este diseño debe garantizar una navegación coherente y accesible no solo durante el transcurso de la partida en el tablero, sino en todo el ecosistema de la plataforma, abarcando la gestión de perfiles, la autenticación y la administración de salas de espera.

- **Objetivo 4. Gestión integral del ciclo de vida del proyecto.**
  Al tratarse de un desarrollo colaborativo realizado por un equipo de dos personas, un objetivo transversal es la aplicación de metodologías ágiles y buenas prácticas de ingeniería. Esto abarca el control de versiones (*Git*), la integración continua, el reparto de tareas mediante tableros *Kanban* y la coordinación eficiente entre el *Frontend* y el *Backend*.

- **Objetivo 5. Despliegue y accesibilidad.**
  Asegurar que el resultado final sea accesible públicamente a través de internet, configurando un entorno de producción real que soporte la concurrencia de usuarios y demuestre la viabilidad técnica de la solución propuesta.

3

Estado del Arte y Marco Tecnológico

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

En este capítulo presenta el estado del arte y el marco tecnológico que sustenta el desarrollo de King and Peasant. En primer lugar, se analizan las soluciones existentes en la digitalización de juegos de mesa, justificando la adopción de un modelo basado en la automatización de reglas frente a la simulación física. A continuación, se detalla y justifica la elección del stack tecnológico, abordando la arquitectura de comunicación mediante WebSockets, el entorno de ejecución asíncrono en Node.js, el renderizado reactivo con React y un sistema de persistencia dual basado en MariaDB y Redis. Finalmente, se expone la estrategia de contenedorización con Docker y la política de pruebas automatizadas para garantizar la calidad y portabilidad del sistema.

3.1 Análisis de soluciones existentes y estado del arte

En la actualidad existen dos acercamientos a la hora de digitalizar un juego de mesa contemporáneo como lo es *King and Peasant*. Por un lado, tenemos el ejemplo de *Tabletop Simulator* (véase la Figura §3.1), que se basa en un entorno de simulación física operando como un çajón de arena"(*sandbox*) [15]. El cumplimiento de las reglas se delega íntegramente a los usuarios, careciendo de una validación lógica que incrementa la carga cognitiva del usuario y ralentiza el flujo de la partida.

Por otro lado, se encuentra *Board Game Arena* (véase la Figura §3.2), que está basado en un motor de validación de estados. En este modelo el cumplimiento de las reglas y las transiciones de estado se delegan al sistema, actuando así como un árbitro imparcial dentro de la partida garantizando la integridad de las mecánicas [6]. El presente TFG se alinea con este segundo enfoque, ya que lo que se busca es crear un motor de reglas estricto que ofrezca una experiencia de juego automatizada, accesible desde el navegador y libre de la fricción que suponen los simuladores tridimensionales

Figura 3.1: Interfaz de *Tabletop Simulator*, basada en la simulación física tridimensional sin validación de reglas.

Figura 3.2: Interfaz de *Board Game Arena*, basada en la automatización del estado de la partida en 2D.

3.2 Arquitectura de comunicación y sincronización

Dada la naturaleza interactiva de un juego multijugador, la sincronización en tiempo real entre múltiples clientes es un punto crítico dentro del desarrollo de *King and Peasant*. Para este propósito, implementar el motor del juego sobre una arquitectura web tradicional resultaba inviable debido a las limitaciones estructurales del protocolo *HTTP*. El uso de técnicas como el *Long-Polling* (*polling*), donde el cliente interroga constantemente al servidor en busca de actualizaciones, introduciría una sobrecarga significativa en la red debido al envío repetitivo de cabeceras HTTP, generando latencias perceptibles que degradan la experiencia de juego [9].

Como alternativa a una API REST, se decidió utilizar el protocolo *WebSocket* como núcleo de las comunicaciones de la partida. Como define el estándar oficial de la IETF [8], esta tecnología proporciona un canal de comunicación bidireccional sobre una única conexión TCP persistente. Esto permite el envío de eventos asíncronos (*server-push*) en el mismo instante en el que se modifica el estado, validando las jugadas y actualizando instantáneamente la interfaz de todos los clientes conectados, reduciendo a su vez el coste computacional y garantizando una sincronización fluida.

3.3 Tecnologías de *Backend* y Ecosistema de Desarrollo

El alto volumen de conexiones concurrentes que presenta un juego multijugador exige un entorno de ejecución capaz de gestionar con eficiencia el consumo elevado de recursos que se produce al mantener las conexiones abiertas en un estado de sincronización permanente. En el caso de los servidores web tradicionales, el modelo de concurrencia basado en hilos (*thread-per-request*) se ve afectado por el cambio de contexto (*context switching*) [13], provocando un rápido agotamiento de la memoria y cuellos de botella. Por otro lado, la mantenibilidad del código y el uso de lenguajes unificados constituyó un factor determinante a la hora de decantarse por una arquitectura de servidor.

Por estos motivos, se adoptó *Node.js*. Su arquitectura basada en un bucle de eventos (*Event Loop*) procesa las solicitudes de red de forma asíncrona, evitando bloquear el hilo principal durante la espera de respuestas de la base de datos o de los clientes. Este diseño arquitectónico permite al sistema mantener activas miles de conexiones bidireccionales de manera simultánea con un consumo residual de recursos. Además, la capacidad de usar lenguajes isomórficos reduce la sobrecarga cognitiva del equipo de desarrollo y permite la compartición directa de módulos lógicos entre cliente y servidor [4].

3.4 Tecnologías de Frontend Reactividad de Interfaz

La implementación de un juego de mesa en un entorno digital multijugador requiere elementos como un tablero interactivo, el cual, en términos de ingeniería, no es más que la proyección visual de una compleja máquina de estados. Sin embargo, se plantea una serie de retos técnicos a la hora de diseñar este componente.

En primer lugar, la manipulación directa del *Document Object Model* (DOM) para actualizar la interfaz resulta computacionalmente ineficiente. El navegador se ve obligado a recalcular estilos y repintar la pantalla cada vez que se produce una modificación, lo que provoca que la fluidez de la experiencia se vea degradada [10]. En segundo lugar, abordar una interfaz que cuenta con tableros, mazos, perfiles y salas de espera bajo una arquitectura monolítica dificultaría enormemente su mantenibilidad y fomentaría la duplicación de código. Finalmente, se le suma el riesgo de desincronización de los estados visuales, resultando en validaciones del *backend* que no se reflejan en la información gráfica [2].

Como solución a todos estos problemas, la arquitectura del cliente se construyó utilizando *React*, una librería que impone un paradigma de desarrollo declarativo y

basado en componentes. En primer lugar, gracias a la implementación de un *Virtual DOM*, la ineficiencia computacional se mitiga. Al recibir un nuevo evento, *React* genera una representación en memoria de la interfaz y la compara con la actual mediante un algoritmo de diferenciación (*diffing*), aplicando al DOM real únicamente las modificaciones estrictamente necesarias. En segundo lugar, su arquitectura modular minimiza el riesgo que supone construir un sistema monolítico, permitiendo dividir la interfaz en unidades lógicas aisladas e independientes, facilitando la encapsulación y garantizando la mantenibilidad a largo plazo del código. Por último, el paradigma declarativo de *React* permite concibir la interfaz matemáticamente como una función del estado subyacente ($UI = f(estado)$) [2], convirtiendo el tablero en un reflejo exacto de la información de la partida y las reglas dictaminadas por el *backend* [10].

3.5 Gestión de la persistencia de datos

Dentro de la información gestionada por el ecosistema encontramos requisitos técnicos independientes, lo que implica la necesidad de implementar un enfoque de persistencia adecuado para cada uno de ellos fundamentado en su naturaleza. Por un lado, la información estructural, como pueden ser las credenciales, perfiles de usuario, historial de partidas o la gestión de salas de espera (*lobbies*), exige un almacenamiento persistente y estructurado que garantice las propiedades ACID (Atomicidad, Consistencia, Aislamiento y Durabilidad) [7]. Para satisfacer esta necesidad, y tras evaluar alternativas relacionales robustas como *PostgreSQL*, se seleccionó el motor de bases de datos relacional *MariaDB* por su agilidad en consultas y su perfecta integración con el ecosistema de *Node.js*. Este motor se encarga de gestionar de forma centralizada la información de los usuarios y la configuración inicial de las partidas. La interacción con este motor relacional se abstrajo mediante *Prisma ORM*, una decisión que previene vulnerabilidades de inyección SQL y garantiza un tipado estricto entre el esquema de la base de datos y la lógica del *backend*.

Por otro lado, el estado del tablero interactivo se convierte en un dato efímero sometido a un volumen masivo de operaciones de lectura y escritura concurrentes. Hacer uso de una base de datos relacional para registrar las modificaciones generaría un cuello de botella debido a la latencia de entrada/salida (I/O), ralentizando la comunicación. Para mitigar este problema, se implementó *Redis* como almacén de estructuras de datos en memoria (*In-Memory Data Grid*) [5]. A diferencia de sistemas de caché simples como *Memcached*, *Redis* soporta el manejo nativo de estructuras de datos complejas, una característica indispensable para modelar el estado de un juego. Hacer uso de la

memoria RAM para almacenar el estado dinámico de la partida permite que el servidor valide y comparta el resultado de las jugadas con tiempos de respuesta mucho menores, delegando en *MariaDB* la persistencia del resultado final una vez concluido el encuentro.

3.6 Contenedorización y automatización de pruebas

El complejo funcionamiento intrínsico del motor de reglas asimétrico requería mecanismos rigurosos para garantizar la calidad y la estabilidad del software. El mal funcionamiento de las validaciones de estado o de las transiciones de la partida degradaría la experiencia de juego. En respuesta, haciendo uso de los marcos de trabajo *Jest* y *Vitest*, se implementó una batería de pruebas automatizadas que aseguraban el correcto comportamiento del motor de validación ante cualquier escenario [11]. Dichas herramientas han permitido aislar y testear unitariamente la lógica del *backend* y los componentes del *frontend*.

De forma paralela, la gestión de la infraestructura se apoyó en la tecnología de contenedores *Docker* [14]. El principal problema que se buscó mitigar con esta adopción fue la inconsistencia de entornos al tratarse de un desarrollo colaborativo entre dos personas. Resultaba necesario aislar los servicios para evitar fallos derivados de las diferencias de *hardware* o configuración del sistema operativo. Por este motivo, se estableció una arquitectura de ejecución híbrida. Por un lado, los servicios de bases de datos (*MariaDB* y *Redis*) se ejecutan exclusivamente mediante contenedores, garantizando que el equipo opere siempre sobre versiones idénticas de estos sistemas. Por otro lado, la capa de aplicación (*frontend* y *backend*) se implementó con un modelo flexible que permite su lanzamiento en local para agilizar la iteración de código, o bien su ejecución contenedorizada. A su vez, esta estandarización de los servicios resulta idónea para simplificar y asegurar el futuro despliegue de la plataforma en entornos de producción.

4

Metodología

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

En este capítulo se define el marco de trabajo y los procedimientos organizativos adoptados para la ejecución del proyecto. Se analiza la estructura del equipo de desarrollo y se realiza un estudio comparativo de distintas metodologías de ingeniería del software, justificando la elección de un enfoque ágil frente a modelos tradicionales. Finalmente, se detalla la adaptación de Scrum a un equipo reducido para garantizar un desarrollo iterativo y eficiente.

4.1 Estructura organizacional del proyecto

El desarrollo de *King and Peasant* ha sido llevado a cabo por un equipo de trabajo constituido por dos ingenieros. Dada la naturaleza reducida del grupo y la fuerte interdependencia técnica entre los componentes del sistema (cliente y servidor), se descartaron las jerarquías verticales en favor de una estructura horizontal y colaborativa.

En lugar de segregar rígidamente los roles, delegando la capa de *Frontend* exclusivamente a un integrante y el *Backend* a otro, ambos miembros del equipo han asumido responsabilidades como desarrolladores *Full Stack*.

Esta decisión arquitectónica a nivel de equipo ha reportado dos beneficios fundamentales para el ciclo de vida del proyecto:

- **Flexibilidad operativa:** Ambos integrantes poseen una visión global y profunda de la arquitectura. Esto ha permitido mitigar cuellos de botella operativos; si un desarrollador no estaba disponible o se encontraba bloqueado en una tarea compleja, el otro poseía el contexto técnico necesario para asistir o continuar el desarrollo sin retrasos.
- **Revisión de código cruzada:** Al conocer ambos la tecnología empleada en todas las capas del sistema, el proceso de validación por pares (*Code Review*) ha resultado mucho más fluido y riguroso. Esta auditoría mutua constante ha sido clave para reducir la introducción de deuda técnica temprana.

A nivel de gestión diaria, la responsabilidad se distribuyó mediante un reparto de tareas dinámico. Al inicio de cada ciclo de trabajo, se evaluaba la complejidad de los requisitos y se asignaban las funcionalidades específicas, garantizando en todo momento un equilibrio equitativo en la carga de trabajo de ambos ingenieros.

4.2 Metodología de desarrollo

Para determinar el modelo de ciclo de vida más adecuado para la naturaleza de este proyecto, se realizó un análisis previo de las metodologías de ingeniería del software más consolidadas en la industria. A continuación, se exponen las alternativas evaluadas y la justificación técnica de la elección final.

4.2.1 Análisis de alternativas tradicionales

Modelo en Cascada (*Waterfall*)

Este enfoque predictivo clásico propone una secuencia puramente lineal de fases (Requisitos, Análisis, Diseño, Implementación, Pruebas y Mantenimiento). Bajo este paradigma, cada etapa debe finalizarse antes de poder comenzar la siguiente.

Motivo del descarte: El desarrollo de un videojuego conlleva una incertidumbre inherente respecto a la jugabilidad y la experiencia final del usuario. El modelo en cascada resulta excesivamente rígido ante esta realidad; si durante la fase de pruebas se detecta que una mecánica de juego presenta fallos de diseño o no resulta atractiva, el coste temporal de retroceder a la fase de análisis para rediseñarla resulta inasumible. Este proyecto requería de una estructura que permitiera pivotar y ajustar requisitos sobre la marcha basándose en el *feedback* interactivo.

Modelo en Espiral

Este modelo evolutivo se centra en la evaluación continua y la mitigación de riesgos mediante iteraciones concéntricas que combinan planificación, análisis de viabilidad y prototipado.

Motivo del descarte: Aunque este paradigma soluciona la rigidez del modelo en cascada, el modelo en espiral introduce una carga de gestión burocrática y de documentación (*overhead*) desproporcionada para un equipo compuesto por solo dos personas. El tiempo invertido en la evaluación formal de riesgos en cada iteración restaría demasiados recursos a la fase de implementación efectiva del código.

4.2.2 Elección: Metodología Ágil (*Scrum*)

Tras descartar los modelos predictivos y evolutivos pesados, la organización del proyecto se decantó por la adopción de una metodología ágil fundamentada en el marco de trabajo *Scrum*. Este enfoque abraza la incertidumbre mediante un desarrollo iterativo e incremental, donde el producto funcional se construye gradualmente en ciclos de tiempo encapsulados.

Esta elección metodológica se justifica por dos pilares clave del proyecto:

1. **Entrega temprana de valor:** Trabajar mediante incrementos funcionales permite consolidar una visión clara del estado del producto desde las primeras semanas

de desarrollo. Esto resulta vital en arquitecturas complejas, ya que permite validar técnicamente la integración entre el cliente y el servidor sin tener que esperar al final del proyecto.

2. **Adaptabilidad y tolerancia al cambio:** Facilita enormemente la comprobación de las reglas del juego, la reestructuración de la interfaz o la alteración de mecánicas tras las pruebas iniciales, todo ello sin desestabilizar la planificación global.

4.3 Adaptación al proyecto: El ciclo de vida ágil

Si bien el marco de trabajo *Scrum* define roles (como el *Scrum Master* o el *Product Owner*) y herramientas diseñadas para equipos de mayor tamaño, en este proyecto se ha ejecutado una adaptación del modelo orientada a la eficiencia. Se ha prescindido de la rigidez burocrática para maximizar la agilidad de un micro-equipo colaborativo.

4.3.1 Sprints Semanales

El esfuerzo de desarrollo se ha dividido en iteraciones temporales cortas o *Sprints* con una duración estricta de una semana. Esta cadencia temporal, notablemente más rápida que las iteraciones de quince días habituales en la industria, ha permitido un nivel de control granular y un seguimiento muy preciso del progreso frente al calendario académico.

El ciclo de cada iteración se ha estructurado en tres eventos fundamentales:

- ***Sprint Planning* (Planificación):** Al inicio de la semana de trabajo, el equipo evalúa y selecciona las tareas prioritarias del *Product Backlog* (pila de producto), estableciendo el objetivo técnico al que se comprometen para esa iteración.
- **Ejecución en paralelo:** Desarrollo efectivo de las funcionalidades y mecánicas acordadas.
- ***Sprint Review* (Revisión e Integración):** Esta validación se lleva a cabo mediante un proceso de revisión cruzada (*Peer Review*), donde cada desarrollador audita el código escrito por el otro antes de autorizar su integración definitiva en la rama principal del repositorio.

4.3.2 Coordinación y herramientas de gestión

Para soportar y dar viabilidad a esta metodología iterativa, el equipo ha empleado herramientas estándar de la industria, asegurando una comunicación constante y la trazabilidad del código:

- **Reuniones de sincronización (*Daily Stand-up*):** Encuentros breves y altamente focalizados (no superiores a 15 minutos) diseñados para coordinar el trabajo de la jornada y, sobre todo, para identificar y mitigar posibles bloqueos técnicos entre ambos desarrolladores.
- **Tablero *Kanban*:** Para la gestión del flujo de trabajo se ha utilizado la herramienta *GitHub Projects*. Este panel permite visualizar en tiempo real el estado evolutivo de las tareas mediante columnas de progreso ("Pendiente", "En curso", "En Revisión", "Terminado").
- **Estrategia de ramificación:** Uso estricto de *Git* para gestionar el ciclo de vida del código fuente. Se ha adoptado una estrategia basada en la creación de ramas independientes por cada nueva funcionalidad (*Feature Branches*), lo cual resulta imprescindible para que dos desarrolladores modifiquen la misma base de código sin generar conflictos críticos de integración.

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

En este capítulo se expone la planificación temporal del proyecto, estructurada mediante la metodología ágil definida previamente. Se detalla el cronograma de ejecución a través de hitos mensuales y Sprints semanales, abarcando desde la configuración inicial de la arquitectura hasta el despliegue final del sistema. Asimismo, se documentan las desviaciones surgidas durante el ciclo de vida del desarrollo y se analizan las medidas correctivas adoptadas por el equipo para garantizar el cumplimiento de los plazos.

5.1 Resumen temporal del proyecto

A nivel global, el proyecto abarca un periodo de desarrollo de cinco meses, desde la configuración inicial de la arquitectura hasta la redacción y entrega de la presente memoria técnica. En la Tabla §5.1 se exponen los parámetros temporales fundamentales.

Resumen del proyecto	
Fecha de inicio	05/01/2026
Fecha de fin	15/05/2026
Periodicidad de las revisiones	1 semanas (Sprints)
Carga de trabajo semanal	12 horas
Horas totales previstas	300 horas
Horas finales	240 horas

Cuadro 5.1: Tabla resumen de tiempos y planificación

5.2 Planificación inicial

Dada la adopción de un *stack* tecnológico complejo y la interdependencia entre el cliente y el servidor, el calendario general se planteó como una hoja de ruta dividida en grandes hitos mensuales. Esta estructura permitió fijar objetivos claros para cada fase del ciclo de vida:

- **Enero - Arquitectura e Infraestructura:** Configuración inicial del repositorio, contenedorización de los servicios mediante *Docker*, implementación del sistema de autenticación de usuarios y salas de espera (*lobbies*), e inicialización del núcleo de la aplicación.
- **Febrero - *Core* del Juego:** Desarrollo íntegro de la lógica de la partida, diseño del tablero interactivo y aseguramiento de la sincronización bidireccional en tiempo real.
- **Marzo - Reglas avanzadas y Seguridad:** Implementación de las mecánicas asimétricas complejas, pulido de la validación de estados en el *backend* y blindaje de la seguridad de la plataforma.
- **Abril - Funcionalidades extendidas y *Testing*:** Integración de características periféricas (como sistemas de interacción social o estadísticas), ejecución exhaustiva

de las baterías de pruebas automatizadas y aseguramiento de la calidad del código.

- **Mayo - Despliegue y Cierre:** Finalización de la presente memoria técnica, despliegue final del sistema y preparación para la defensa académica.

La determinación de estas fechas no ha sido arbitraria, sino que responde a un análisis de la carga lectiva y los hitos del calendario académico de la *Escuela Técnica Superior de Ingeniería Informática*. Se estableció un ritmo de trabajo sostenido y diario con el objetivo de compatibilizar el desarrollo del proyecto con el resto de asignaturas del cuarto curso. Esta cadencia diaria ha sido la herramienta principal para evitar la acumulación de tareas en periodos de exámenes, permitiendo que el avance del proyecto fuera independiente de las fluctuaciones de carga de trabajo de otras materias.

5.2.1 Cronograma de iteraciones (Nivel Micro)

Para materializar los hitos mensuales, el trabajo se fragmentó en *Sprints* semanales. En la Tabla §5.2 se detalla la planificación inicial extraída del tablero Kanban, mostrando los objetivos técnicos asignados a cada periodo. Cabe destacar que todas las iteraciones incluyen de forma implícita tareas de documentación continua y reuniones de *Sprint Review & Planning*.

Mes	Semana (Sprint)	Objetivos Técnicos Principales
Enero	12 - 18 Ene	Inicialización de repositorio, Set Up Docker, Frontend/Backend y bases de datos. Diagramas UML.
	19 - 25 Ene	Set Up Despliegue Continuo, Tests, Metodología Git y Mockups UI.
	26 - 1 Feb	Desarrollo de Pantalla de Inicio y Login/Registro.
Febrero	2 - 8 Feb	Pantalla de Listado de Salas.
	9 - 15 Feb	Pantalla de Usuario (Editar), Sistema de Amistad y Pantalla de Sala.
	16 - 22 Feb	Funcionalidad invitar usuario, Pantalla Partida Prueba y Chat de Partida.
	23 - 1 Mar	Sistema de Barajas/Cartas, Inspección de roles y Flujo de estados/turnos.
Marzo	2 - 8 Mar	Accionar cartas, Temporizador, UI extra de partida y Acciones Simples.
	9 - 15 Mar	Set up inicio partida, Fin de Era/Partida y Condiciones finales.
	16 - 22 Mar	Implementación de Acciones Complejas 1.
	23 - 29 Mar	Implementación de Acciones Complejas 2 y Pantalla de Ganador.
Abril	30 - 5 Abr	Mover cartas (*Drag & Drop*) y Animaciones de cartas.
	6 - 12 Abr	Test Lógica de Juego 1, Test Sala y Test Usuarios.
	13 - 19 Abr	Test Lógica de Juego 2, Rediseño estético final.
	20 - 3 May	Refactorización general y solución de errores (*Bugfixing*).
Mayo	4 - 10 May	Pruebas finales e integración completa.
	11 - 15 May	Finalización y revisión de la memoria técnica.

Cuadro 5.2: Cronograma inicial de tareas distribuidas por Sprints semanales.

5.3 Gestión de desviaciones y medidas correctivas

A pesar de la planificación inicial, el desarrollo de un proyecto de ingeniería está sujeto a imprevistos técnicos y personales. Siguiendo las directrices de una metodología ágil, el equipo ha demostrado capacidad de adaptación mediante la aplicación de medidas correctivas inmediatas ante las desviaciones ocurridas.

5.3.1 Incidencia técnica: Fallo en el hardware de desarrollo

Durante el mes de febrero, coincidiendo con el desarrollo del *core* del sistema, uno de los miembros del equipo sufrió una interrupción crítica debido a un fallo técnico en su estación de trabajo (problemas de almacenamiento que derivaron en la inoperatividad del sistema operativo). Esta incidencia supuso la pérdida de una semana de capacidad de desarrollo para dicho integrante.

Para mitigar el impacto en el cronograma global y evitar un retraso en la entrega del *Sprint*, se aplicaron las siguientes medidas de contingencia:

- **Redistribución dinámica de la carga:** El segundo integrante asumió las tareas críticas que se encontraban en la ruta principal de esa iteración, asegurando que el flujo de trabajo del servidor no se detuviera.
- **Sincronización y continuidad:** Una vez restaurado el entorno de trabajo, el miembro afectado pudo retomar su actividad de forma inmediata sobre la base de código ya avanzada por su compañero. La estructura horizontal del equipo y el conocimiento *Full Stack* compartido permitieron que esta transición fuera transparente y sin pérdida de contexto técnico.

Como resultado, la desviación temporal fue absorbida por el propio equipo sin afectar a la fecha de finalización del hito mensual, validando la eficacia de la estructura organizacional elegida.

5.3.2 Conciliación académica y personal

Otra desviación potencial fue la gestión del tiempo durante los picos de carga de trabajo académico. La medida correctiva aplicada fue la intensificación de las reuniones de sincronización (*Daily Stand-ups*) para ajustar los objetivos del *Sprint* de forma realista semana a semana. El compromiso con un flujo de trabajo constante ha permitido que, a

pesar de las situaciones personales o de estudio, el proyecto no haya sufrido periodos de inactividad, manteniendo una entrega de valor continua desde el inicio del semestre.

31

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

6.1 Resumen de costes del proyecto

Resumen del proyecto	
Costes de personal	5.045 €
Sueldo neto	2.030 €
Impuestos	1.000 €
Costes sociales	2.015 €
Costes materiales	560 €
Costes indirectos	450 €
TOTAL	8.000 €

Cuadro 6.1: Tabla resumen de costes

6.2 Costes de personal

Ya hablaremos de esto

6.3 Costes materiales

Y de esto también. Ver Sección §7.4.

6.4 Costes indirectos

Y esto es una fiesta

7

isis y Diseño del Sistema

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

En este capítulo se definen los cimientos funcionales y técnicos del proyecto. En primer lugar, se establece el catálogo de requisitos funcionales y no funcionales, delimitando las fronteras del sistema. A continuación, se modela la interacción de los usuarios mediante diagramas de Casos de Uso y de Secuencia. Finalmente, se describe el diseño de la arquitectura de software y el modelo de datos diseñado para garantizar el rendimiento en tiempo real.

7.1 Análisis de Requisitos

La fase de análisis tiene como objetivo definir el contrato del sistema, estableciendo de forma unívoca qué debe hacer la aplicación (requisitos funcionales) y bajo qué restricciones de calidad, rendimiento y seguridad debe operar (requisitos no funcionales).

7.1.1 Requisitos Funcionales y Actores

Para la definición del Producto Mínimo Viable (MVP) de *King and Peasant*, se han identificado dos perfiles de interacción o **Actores** principales:

- **Usuario No Autenticado (Invitado):** Entidad que interactúa con las capas públicas del sistema, con el único objetivo de registrarse o autenticarse.
- **Jugador (Usuario Registrado):** Actor principal que posee una sesión activa y tiene acceso a la totalidad de las mecánicas del juego y módulos sociales.

Las características esenciales del sistema se estructuran en los siguientes módulos funcionales:

1. **Gestión de Identidad:** Registro, inicio de sesión seguro y gestión del perfil del usuario (estadísticas e historial).
2. **Ecosistema Social:** Sistema de peticiones de amistad y chat en tiempo real.
3. **Infraestructura de *Lobbies*:** Creación de salas de juego (públicas y privadas) y sistema de emparejamiento.
4. **Motor de Juego (*Core*):** Lógica asimétrica de cartas, gestión de turnos, temporizadores y sincronización bidireccional del estado del tablero.

7.1.2 Requisitos No Funcionales (RNF)

Dada la naturaleza interactiva del proyecto, los requisitos no funcionales resultan críticos para la viabilidad del sistema. Se han definido las siguientes restricciones técnicas:

- **RNF-ARQ (Arquitectura y Stack):** El cliente debe construirse como una *Single Page Application* (SPA) con *React* 18 y *TypeScript*. El servidor utilizará *Node.js* (Express) bajo una arquitectura modular. El código fuente se organizará estrictamente como

un Monorepositorio mediante *NPM Workspaces* para compartir dependencias entre cliente y servidor.

- **RNF-REN (Rendimiento):** El sistema exige **baja latencia**. Las acciones de la partida deben procesarse en tiempo real utilizando el protocolo *WebSockets* (*Socket.io*) y apoyándose en *Redis* para el estado volátil. Asimismo, todo el entorno debe estar contenedorizado con *Docker* para garantizar un despliegue y escalabilidad ágiles.
- **RNF-SEG (Seguridad):** La autenticación se basará en *JSON Web Tokens* (JWT). Las credenciales se protegerán mediante algoritmos de *hashing* unidireccional (`bcryptjs`) y se aplicarán políticas CORS estrictas. Los secretos del sistema se aislarán mediante variables de entorno (`.env`).
- **RNF-FIA (Fiabilidad y QA):** Se requiere un flujo de Integración y Despliegue Continuo (CI/CD) automatizado. La calidad del código se garantizará mediante pruebas unitarias en el *backend* (*Jest*) y en los componentes del *frontend* (*Vitest*).

7.2 Modelado del Sistema

Para representar gráficamente las interacciones descritas en los requisitos, se ha utilizado el Lenguaje Unificado de Modelado (UML), estandarizando la comprensión de la arquitectura.

7.2.1 Diagrama de Casos de Uso

En la Figura §7.1 se detalla el comportamiento del sistema desde el punto de vista del usuario. Se observa cómo el *Jugador* hereda la capacidad de acceder al ecosistema central tras superar los casos de uso del *Invitado*. Destaca la modularidad del caso de uso principal ("Jugar Partida"), el cual incluye («*include*») obligatoriamente la mecánica de jugar cartas, y se extiende («*extend*») de forma opcional con acciones como el chat en vivo o la rendición voluntaria.

Figura 7.1: Diagrama General de Casos de Uso del sistema.

7.2.2 Comportamiento Dinámico: Flujo de Ejecución

Dado que los casos de uso estáticos no muestran el orden cronológico de las operaciones, se ha elaborado un Diagrama de Secuencia (Figura §7.2) que ilustra la travesía completa de un usuario: desde que no tiene cuenta hasta que juega una carta.

El diagrama evidencia la transición entre el protocolo HTTP y el protocolo WebSockets. Mientras que el registro y la creación de la sala (*Lobby*) se resuelven mediante peticiones REST tradicionales que persisten en *MariaDB*, el inicio de la partida desencadena una actualización en la base de datos en memoria (*Redis*). A partir de ese punto, acciones iterativas como enviar un mensaje o mover una carta se transmiten mediante eventos asíncronos (*Broadcast*), logrando la latencia nula exigida en los requisitos no funcionales.

Figura 7.2: Diagrama de Secuencia: Registro, creación de sala y flujo de partida.

—

7.3 Lista de características

Siguiendo la metodología de desarrollo iterativo, se ha definido un conjunto de características esenciales (*features*) que constituyen el Producto Mínimo Viable (MVP) de *King and Peasant*. Estas características se han agrupado por módulos funcionales:

- **Gestión de Identidad y Usuarios:**
  - Registro de nuevos usuarios con validación de credenciales.
  - Inicio de sesión seguro (*Login*) mediante JWT y gestión de sesiones.
  - Perfil de usuario con visualización de avatar y estadísticas básicas.
- **Infraestructura de Salas (Lobby):**
  - Creación de salas de juego públicas y privadas.
  - Listado de salas activas en tiempo real.
  - Capacidad de unirse a una sala existente mediante código o selección directa.
- **Motor de Juego (Core):**
  - Inicialización de la partida: barajado y reparto de cartas.
  - Gestión de turnos: control de tiempos y validación de acciones permitidas por jugador.
  - Mecánicas de cartas: lógica de ataque, defensa y efectos especiales.

    - Sincronización de estado: actualización visual del tablero para todos los participantes simultáneamente.

- **Funcionalidades Sociales:**
    - Chat de texto en tiempo real dentro de la sala de espera y durante la partida.
    - Sistema de lista de amigos con gestión de estados (conectado/desconectado) e invitaciones.

7.4 Diseño arquitectónico y Organización

La arquitectura del sistema se ha diseñado siguiendo un patrón cliente-servidor desacoplado. Para gestionar la complejidad de dos bases de código interrelacionadas, se ha adoptado una estrategia de **Monorepo** (Monorepositorio) utilizando los *Workspaces* de `npm`.

Esta estructura permite mantener tanto el código del cliente ('packages/client') como el del servidor ('packages/server') en un único repositorio, facilitando la orquestación de scripts de desarrollo, la gestión de dependencias comunes y el despliegue unificado mediante contenedores.

7.4.1 Arquitectura del Backend (Servidor)

El servidor, desarrollado en **Node.js** con **Express**, no es monolítico, sino que implementa una **Arquitectura por Capas** (*Layered Architecture*) para garantizar la separación de responsabilidades y la escalabilidad.

1. **Capa de Enrutamiento (Routes):** Define los puntos de entrada de la API REST (por ejemplo, '/api/lobby', '/api/game'). Su única función es recibir la petición HTTP y delegarla al controlador adecuado.
2. **Capa de Controladores (Controllers):** Gestiona la interacción con el cliente. Valida los datos de entrada, invoca a los servicios necesarios y formatea la respuesta JSON adecuada.
3. **Capa de Servicios (Services):** Contiene la **lógica de negocio pura**. Es el núcleo del sistema.

- Por ejemplo, el servicio 'FriendshipService.js' encapsula la lógica compleja para verificar relaciones bidireccionales o filtrar solicitudes pendientes, aislando esta complejidad de la capa HTTP.

4. **Capa de Persistencia y Datos:** Se ha optado por un modelo híbrido para optimizar el rendimiento:

   - ***MariaDB + Prisma ORM*:** Utilizada para datos persistentes (usuarios, historial). *Prisma* permite interactuar con la base de datos mediante objetos tipados, previniendo errores de sintaxis SQL y facilitando las migraciones del esquema.
   - ***Redis*:** Base de datos en memoria clave-valor. Es fundamental para almacenar el estado volátil de las partidas y las sesiones de usuario, garantizando una latencia mínima crítica para el juego.

Gestión de Tiempo Real (WebSockets) Paralelamente a la API REST, el servidor gestiona conexiones persistentes mediante `socket.io`. Se ha implementado una estructura en memoria ('Map') que vincula los IDs de usuario de la base de datos con sus conexiones de *socket* activas. Esto permite funcionalidades avanzadas como enviar invitaciones privadas específicas o gestionar desconexiones abruptas (rendición automática) en tiempo real.

7.4.2 Arquitectura del Frontend (Cliente)

El cliente es una *Single Page Application* (SPA) desarrollada con **React** y construida sobre **Vite** para maximizar la velocidad de desarrollo. A diferencia del servidor (JavaScript), el cliente está escrito íntegramente en **TypeScript**, lo que aporta robustez y seguridad de tipos en la manipulación del estado del juego.

La organización del código cliente sigue principios de modularidad:

- **Gestión de Estado Global (Context API):** Se utiliza un 'AuthProvider' para encapsular la sesión del usuario y distribuirla a toda la aplicación sin necesidad de propagar propiedades manualmente (*prop drilling*).
- **Hooks Personalizados:** La lógica de negocio del cliente se extrae a *hooks* propios (como 'useUser' o 'useAuth'), separando la lógica funcional de los componentes visuales (`.tsx`).

- **Calidad de Código:** Se utiliza **Vitest** para la ejecución de pruebas unitarias de componentes y lógica, asegurando que la interfaz responde correctamente a los cambios de estado del juego.

7.4.3 Infraestructura y Despliegue

La consistencia entre los entornos de desarrollo y producción se garantiza mediante la contenerización completa del sistema.

Entorno de Desarrollo (Local)

El entorno local está orquestado con **Docker Compose**, lo que permite a cualquier miembro del equipo levantar la infraestructura completa con un solo comando. El archivo 'docker-compose.yml' coordina:

- Un contenedor para ***MariaDB***.
- Un contenedor para ***Redis***.
- El contenedor del servidor (*Node.js*) configurado con recarga en caliente (*hot-reload*) para reflejar cambios inmediatos en el código.

Entorno de Producción

Para el despliegue público, se utiliza un servicio en la nube (PaaS) que ejecuta los contenedores optimizados. La seguridad se refuerza mediante el uso de certificados SSL/TLS, cifrando tanto las peticiones HTTPS como el tráfico WSS (*Secure WebSockets*), protegiendo así la integridad de las partidas y los datos de los usuarios.

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

8.1 Características a desarrollar

1. Funcionalidad A. Ver Tabla §8.1.
2. Funcionalidad B.

Análisis de valor aportado 0001	
Propuesta	Trabajo que pretende analizarse y justificarse
Valor	Qué valor aporta al proyecto o al usuario final.
Coste	Qué costes en términos de esfuerzo, adquisiciones y limitaciones tiene la propuesta
Opciones	Qué otras opciones se tienen que aporten un valor similar? ¿Es realmente un valor relevante para el proyecto/cliente
Riesgos	Qué riesgos pueden surgir a la hora de desarrollar esta propuesta.
Deuda técnica	Posibles deudas técnicas que se asumen con el desarrollo de esta propuesta.

Cuadro 8.1: Análisis de valor aportado 0001

8.2 Diseño

Aquí una discusión de cómo va a afectar todo al diseño

Debe insertarse un diagrama UML de diseño con los cambios y hacer referencia en el texto así Fig. §8.1.

Un memorando técnico por cada decisión de diseño.

8.3 Implementación

Un memorando técnico por cada decisión de implementación y refactorización que afecte al diseño.

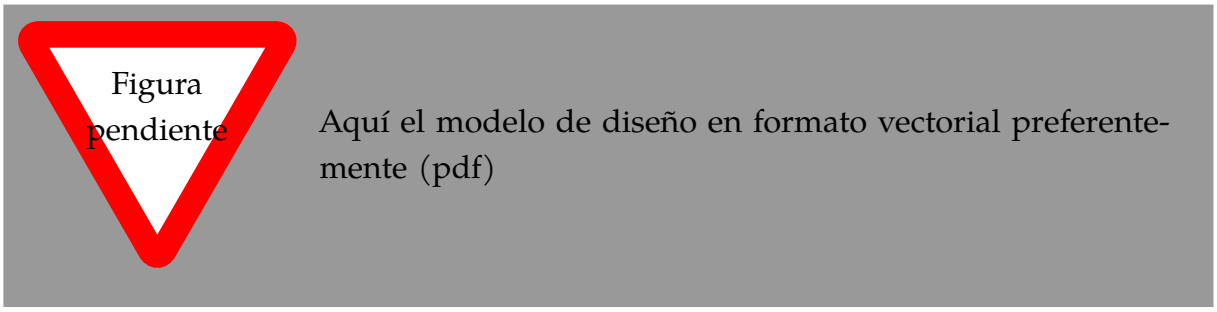
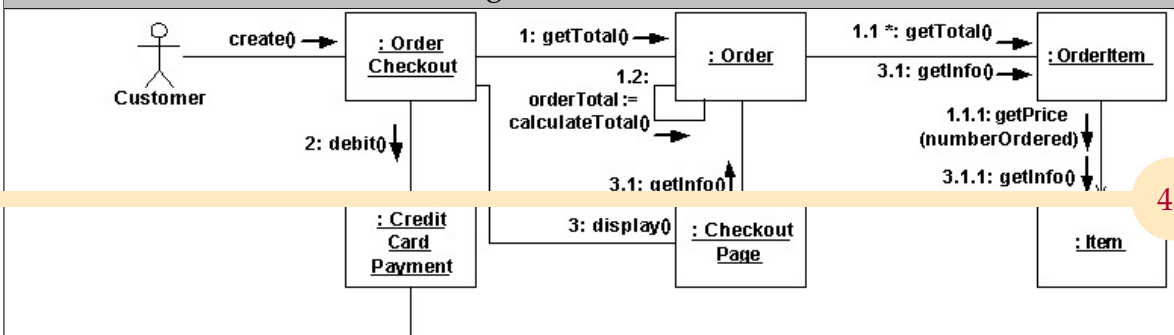


Figura 8.1: Diagrama UML de diseño para la iteración 1

Memorando técnico 0001	
Asunto	¿Cuál es el problema?
Resumen	¿Cuál es la solución propuesta?
Factores causantes	Descripción pormenorizada del problema
Solución	Descripción pormenorizada de la solución propuesta
Motivación	¿Por qué propone esta solución?
Cuestiones abiertas	Factores a tener en cuenta en la solución cuya dimensión se reconoce.
Alternativas	Otras soluciones consideradas y la razón por la que se excluyeron.

Cuadro 8.2: Memorando técnico 0001

Identificador	Descripción de la acción de alto nivel			
0001	Prueba			
Métodos de alto nivel				
[return_type] method_name1 (param1:type1, ...)				
Pasos (Usar Pseudocódigo o similar)				
1. Paso 1.				
2. Paso 2.				
Métodos de bajo nivel necesarios				
Paso	**Clase**	**Método**	**Mem. Técn.**	**IU**
1	ClassName	[return_type] method_name1 (param1:type1, ...)	001	SI
Diagrama de Colaboración				



Identificador	Descripción de la acción de alto nivel
alvotermar02	Grubber

Métodos de alto nivel
[return_type] grubber (param1:type1, ...)
Pasos (Usar Pseudocódigo o similar)
1. Lanzar 2 dados
2. Compara resultado de los dados con kicking del open-side
2.1. Si valor dados es menor o igual a kicking, avanza 10m
3.1. Si no hay defensa y el golpeo es exitoso, el pateador retiene la posesión del balón
3.2. Si hay defensa y el golpe es exitoso, el atacante tira un dado y suma su valor al de speed y strength y el defensor lanza 2 dados y lo suma al valor de speed y strength de su jugador, el vencedor será aquel que tenga más puntos, si es igual, la posesión es del defensor
4.1. Si no es exitoso y hay defensa el balón pasa a posesión del defensor
4.2. Si no es exitoso y no hay defensa de lanza un line-out

Métodos de bajo nivel necesarios				
Paso	**Clase**	**Método**	**Mem. Técn.**	**IU**
1	Dice	[Integer] throwDice ()	001	SI
2	ClassName	[Int] compareKickingToDice (kicking:Integer, dice: Integer)	001	SI
2.1	ClassName	[Integer] setLine (line:Integer)	001	SI
4.2	ClassName	[Integer] lineOut ()	001	SI

8.4 Pruebas

Descripción de las pruebas realizadas al software

8.5 Despliegue

Breve resumen de cómo se han desplegado los cambios en el sistema de producción.

Manual de usuario

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

9.1 Sección libre

Estructurar en función del proyecto.

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

Resumen de lo que va a ocurrir en el capítulo. ¿Cuál es el objetivo que tenemos con este capítulo?

10.1 Informe post-mortem

Qué es un informe post-mortem

10.1.1 Lo que ha ido bien

- Argumento a favor 1.
- Argumento a favor 2.
- Argumento a favor 3.

10.1.2 Lo que ha ido mal

- Argumento en contra 1.
- Argumento en contra 2.
- Argumento en contra 3.

10.1.3 Discusión

En función de lo anterior, qué cambiaría si empezara hoy el proyecto de nuevo.

10.2 Trabajos futuros

Enumera los puntos abiertos y que no se han resuelto. Indica si darían lugar a otro proyecto y de qué forma se podría acotar.

Parte V

Appendices

The good parts of a book may be only something a writer is lucky enough to overhear or it may be the wreck of his whole damn life – and one is as good as the other.

Ernest Hemingway (1899–1961),
Novelist

This is an example of an abstract. Multiple lines are supported. Several paragraphs. It jumps to the next page. Blau blau blau. I am introducing more text to reach the third line

A.1 Software Product Lines

- Objective of a *Product Line* (*PL*) (mass production and customisation) [3]
- The focus in software derives in *Software Product Lines* (*SPLs*).
- Variability management: variability models
- When and how are used VMs: FMs are described in FODA report as a key element in SPL since they represent the variability and commonality of the different products in a SPL.

A.2 Feature Models

To Abductive Section in 2.1

As the number of products to be built by a SPL may be large and the constraints among features may be complex, representing such an information in a manageable and compact manner is a must. *Feature Models* (*FMs*) represent the set of products a SPL may build in terms of product features. Some features are optional while others are mandatory. To indicate the relationships among features, they are hierarchically linked, forming a tree whose root is a feature representing the whole functionality of a product. The root feature is refined in child features, which increase the level of detail and reduce the scope of features. Recursively following this refinement process, a tree-like structure is obtained where three basic kinds of hierarchical relationships are used:

- Mandatory: a mandatory relationship affects a parent and child feature. It forces the child feature to appear in a product whenever its parent feature does.
- Optional: a child feature connected to a parent feature by means of an optional relationship may be optionally selected whenever its parent feature is.
- Set-relationships: three or more features are part of a set-relationship: a parent feature and a set of two or more child features. A set-relationship contains a cardinality that constraints the number of child features to be selected in a product whenever its parent feature is selected. If the cardinality is $[1.,1]$ it is commonly remarked as an *alternative relationship* where only one child feature may be selected at the same time. If the cardinality is $[1..N]$ (where N is the number of child

features), it is also known as an *or-relationship* as any combination of child features is allowed while at least one is selected.

Although FMs can represent most of the most frequent constraints, the hierarchical nature of these models might hinder the representation of some constraints. Under this circumstance, *cross-tree constraints* can be added. The most common kinds of cross-tree constraints are:

- Dependency: a feature depends on another feature if the second one must be part of a product whenever first one is selected.
- Exclusion: two features exclude themselves if both of them cannot be part of a product at the same time.

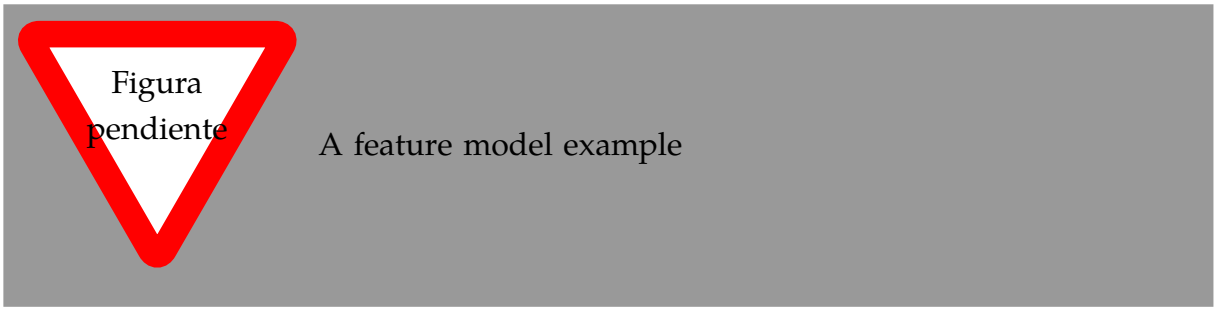


Figura A.1: An example of a Home Integration System

The example in Figure §A.1 describes a *Home Integration System* (HIS) SPL in terms of its features and the relationships among them. Leaning on this example we define some useful terms:

Partial configuration : a partial configuration is a composed by three sets of selected (S), removed(R) and undecided(U) features. A feature can only be in one of these sets and every feature in the FM (fm) must be in one of them, i.e. $S \cup R \cup U = fm$ and $S \cap R \cap U = \emptyset$. A partial configuration represents an intermediate state during the process of a customer selecting the feature for a custom product. For example, $S_P = \{...\}$, $R_P = \{...\}$ and $U_P = \{...\}$ define a partial configuration for the sample FM where some features are still to be decided if they are to be selecter or removed in a configuration.

(Full) configuration : a full configuration or simply a configuration is a partial configuration such that the set of undecided features in empty. For example, $S_F = \{...\}$ and $R_F = \{...\}$ describe a full configuration for the example FM.

Product : a product is a representation for a full configuration such that only the selected features are remarked. For instance, $P = \{\}$ is a product for the above full configuration. A product such as `A,B` is a valid since all the constraints within the FM are satisfied. However, A,B and C is not a valid product since D is required.

Validation A partial configuration is *valid* if all the relationships and constraints are satisfied given the sets of selected, removed and undecided features. So the definition applies for valid full configurations and valid products. As a conclusion we can affirm that a FM represents all the valid products in a SPL.

Objetivo: Briefly expose attributes as an important asset in feature models.

It is frequent that features are not enough to represent information that is relevant to represent a SPL variability. In this case, FMs are extended with feature attributes such as cost, versions, RAM consumption, etc. in the so-called *Extended Feature Models* (*EFMs*) [3]. Besides relationships, an EFM contains constraints that affect attributes which reduce even more the set of products a FM describes. Above definitions remain when attributes are introduced into FMs.

A.3 Automated Analysis of Feature Models

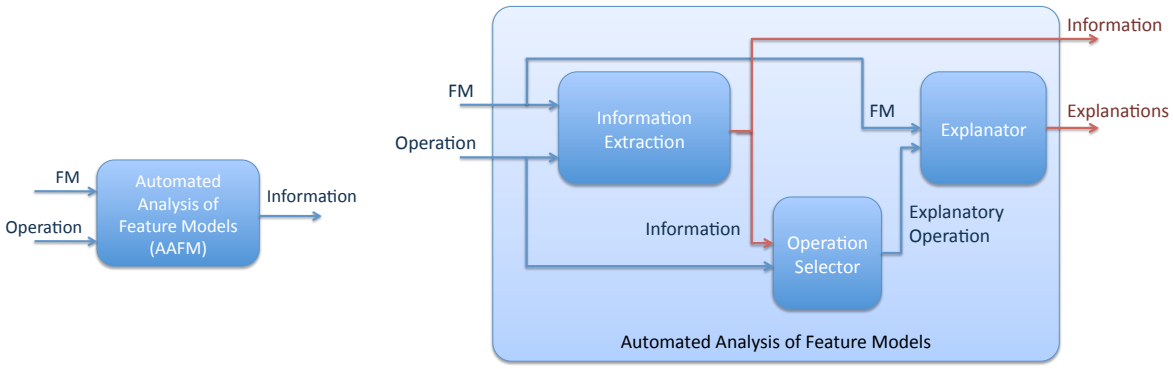
A.3.1 Scope

To Abductive Intro

FMs are used all along the SPL development as key models and many of the development decisions are taken relying on the information contained within them. Most of the times, relationships are complex and hinder the manual extraction of information. Manually obtaining information such as 'which is the product that costs the less?', 'does the feature model contain errors?' or 'why there exist no product containing certain features?' can be an unfeasible task. The complexity and compactness of FMs justify the need of an automated support of these operations. So the *Automated Analysis of Feature Models* (AAFM) arises as a topic of interest to deal with this problem in the SPL community.

The AAFM can be seen as a black-box process that receives a FM and an operation as inputs and obtains information (its kind depends on the analysis operation) as an

output (Fig. A.2(a)). There are many operations that extract information from a FM such as 'counting products' operation whose result is a natural number indicating the number of customised products that can be built; or 'list of products' operation that obtains each of those products. This vision of AAFM as a black-box is valid for a subset of analysis operations that we call *information extraction operations* (IEO) that can be seen as processes to extract information from FMs. In other words, an IEO makes explicit an implicit information within a FM.



(a) The AAFM seen as a black-box process

(b) Extending the AAFM process with explanations

Figura A.2: A different view on AAFM distinguishing between information extraction and explanatory operations

Use me to explain in a larger text than 'sidetext' anything that is important to a reader not familiar with the dissertation context for example.

However, there is a subset of analysis operations known as *explanatory operations* (EO) whose objective is explaining the result obtained from a IEO. Sometimes, the result is not the expected one and the analyser needs to know which are the relationships that have caused it. For example, let us suppose that the IEO 'which are the products described in a FM that cost less than $1000?' obtains no products as a result. If we were expecting to obtain at least one product, it is important to determine the relationships in the FM that are responsible of that behaviour, so an EO 'why there is no product costing less than $1000?' will shed light on the relationships that avoid obtaining any product. Obtaining no result is not the only case that claims for explanations. If we obtained only one product as a result and we were expecting to obtain at least 10 products, although an answer is obtained the result is unexpected and the discrepancy reasons have to be found. Moreover, explanatory operations are also useful even when an expected result

is obtained, to reinforce the certainty that the result is correct. So it can be concluded that EOs complement the information an FM analyser obtains from IEOs.

The complexity of feature modelling relies on correctly setting the relationships that describe the set of products to be built in a SPL. Relationships are the only elements responsible of the results obtained in FM analysis. So an *explanation* is a set of relationships that may have caused that result. While IEO provides for an unique response that is known for certain, an EO provides for a set of probable explanations to a result obtained from a IEO, being only one of them a valid explanation. It would be the analyser the one in charge of discriminating the correct explanation, maybe performing new analysis operations.

THIS IS A SIDE TEXT. USE TO REMARK IMPORTANT INFORMATION

Therefore, two kinds of operations are distinguished in AAFM: information extraction and explanatory operations. Explanatory operations have no sense without a paired information extraction operation and its result. To ensure that explanatory operations are always paired to an information extraction operation, we define a new black-box process of AAFM that incorporates explanations as an additional output (see Figure A.2(b))

1. Information extraction: the original process, which remains the same.
2. Operation selector: depending on the information extraction operation the analyser asks for and the information obtained as a result, this process provides the explanatory operation to be performed. In other words, it pairs an explanatory operation to an information extraction operation.
3. Explanatory analysis: provides a set of explanations from the FM and the explanatory operation.

The overall process can be encapsulated into a holistic black-box process which receives the FM and the information extraction operation as inputs and provides a result and explanations as outputs. It can be seen as we just add explanations as an output to the analysis process.

To realise this view on the AAFM, we need to get details on the insides of these black-boxes. Since the information extraction process is already rigourously defined in Benavides' PhD dissertation, the purpose of this paper is defining the remaining two

sub-processes. We formalise the explanatory analysis process by means of default logic and provide the criteria to implement the operation selector process.

Most Common Techniques to perform AAFM Operations.

A.4 Dynamic Software Product Lines (DSPL)

What is a *Dynamic Software Product Line* (*DSPL*). Different points of view. What is important is the automation of reconfiguration properties relying on SPL techniques.

We focus in the application of explanations in DSPLs as an application of our results. Specifically we have worked in MAS and smart homes providing a solution for automating product reconfiguration.

A.5 Hypothesis and Objectives

Objetivo: Justifying that explanations are a particular set of operations in AAFM that are not solvable by means of the techniques that are used up-to-date

Objetivo: Set an impacting phrase that summarises the hypothesis

Hypothesis

Explanations cannot be solved by AI techniques used to solve AAFM. There should exist other AI techniques to solve explanations.

Objective of the dissertation

Defining a framework to provide solutions for explanatory analysis in FMs.

This dissertation summarises our contribution to solve some of the objectives we set in our PhD project.

- Defining a catalog of analysis operations where explanations are applied.
- Rigorously defining these operations in terms of logics.
- Proposing solutions to these operations.
- Validating our results by means of tools and projects where they are applied.

Next chapter focuses on refining how we have contributed to deal with the above objectives.

A piece of code...

```
public Map<Cardinality,CardinalValue> detectWrongCardinals() {
        // any other implementation of Map can be used instead.
        Map<Cardinality,CardinalValue> result =
                new TreeMap<Cardinality,CardinalValue>();
        for( r : relationships) {
                if (r instanceof Set) {
                        Set set = (Set)r;
                        Cardinality card = set.getCardinality();
                        Domain dom = card.getDomain();
                        for (value: dom.getValues())
                                if (isWrongCardinal(card,value))
                                        result.put(card,value);
                }
        }
        return result;
}
```

A coolTable. Use inside a table.

Use \TableSubtitle{n,title} to add a subtitle as the header. n is the number of columns and title is the text to place. [3]

A Catalog of FM Explanatory Operations (2009 version)		
Information Extraction Operation	**FM Explanatory Operations**	
	Why? operation	*Why not? operation*
Valid FM	-	invalid FM
Valid Configuration	valid partial conf.	invalid partial conf.
Valid Product	valid product	invalid product
Products Listing	vaild Product/Config	invalid FM/Product/Config
Products Counting	vaild Product/Config	invalid FM/Product/Config
Optimisation	vaild Product/Config	invalid FM/Product/Config
Core feature	core feature	core feature
Variant feature	variant feature	variant feature
Dead feature detection	-	dead feature
False-optional feature detection	-	false-optional feature
Wrong-cardinality detection	-	wrong cardinal
Information Extraction Operation	**Configuration Explanatory Operations**	
	Why? operation	*Why not? operation*
Valid Configuration	valid partial conf.	invalid partial conf.

Cuadro A.1: Most frequently used explanatory operations and their corresponding information extraction operations

[1] Adobe Systems. Flash & the future of interactive content. https://theblog.adobe.com/flash-the-future-of-interactive-content, 2017. Anuncio oficial del fin de vida (EOL) de Flash Player. Accedido: 2026-02-18.

[2] A. Banks and E. Porcello. *Learning React: Modern Patterns for Developing React Apps*. O'Reilly Media, Inc., 2nd edition, 2020. ISBN 9781492051225.

[3] D. Benavides, A. Ruiz-Cortés, and P. Trinidad. Automated reasoning on feature models. *LNCS, Advanced Information Systems Engineering: 17th International Conference, CAiSE 2005*, 3520:491–503, 2005. ISSN 0302-9743.

[4] M. Cantelon, M. Harter, T. J. Holowaychuk, and N. Rajlich. *Node.js in Action*. Manning Publications Co., 2013. ISBN 9781617290572.

[5] J. L. Carlson. *Redis in Action*. Manning Publications Co., 2013. ISBN 978-1617290855.

[6] G. S. Elias, R. Garfield, and K. R. Gutschera. *Characteristics of Games*. MIT Press, 2012. ISBN 978-0262017138.

[7] R. Elmasri and S. B. Navathe. *Fundamentals of Database Systems*. Pearson, 7th edition, 2015. ISBN 978-0133970777.

[8] I. Fette and A. Melnikov. The websocket protocol. RFC 6455, RFC Editor, 2011. URL https://www.rfc-editor.org/info/rfc6455.

[9] S. Loreto, P. Saint-Andre, S. Salsano, and G. Wilkins. Known issues and best practices for the use of long polling and streaming in webrtc, xmlhttprequest and server-sent events. RFC 6202, RFC Editor, 2011. URL https://www.rfc-editor.org/info/rfc6202.

[10] A. Mardan. *React Quickly: Painless web apps with React, JSX, Redux, and GraphQL*. Manning Publications Co., 2017. ISBN 9781617293344.

[11] G. J. Myers, C. Sandler, and T. Badgett. *The Art of Software Testing*. John Wiley & Sons, 3rd edition, 2011. ISBN 978-1118031964.

[12] nPipsGames. King and peasant, 2024. URL https://www.npipsgames.com/king-peasant. Accedido: 28 de abril de 2024.

[13] S. Tilkov and S. Vinoski. Node.js: Using javascript to build high-performance network programs. *IEEE Internet Computing*, 14(6):80–83, 2010. doi: 10.1109/MIC.2010.145.

[14] J. Turnbull. *The Docker Book: Containerization is the new virtualization*. James Turnbull, 2014. ISBN 978-0988820203.

[15] S. Woods. *Eurogames: The Design, Culture and Play of Modern European Board Games*. McFarland, 2012. ISBN 978-0786467976.